Web Programming 2 (C#)

Week 4

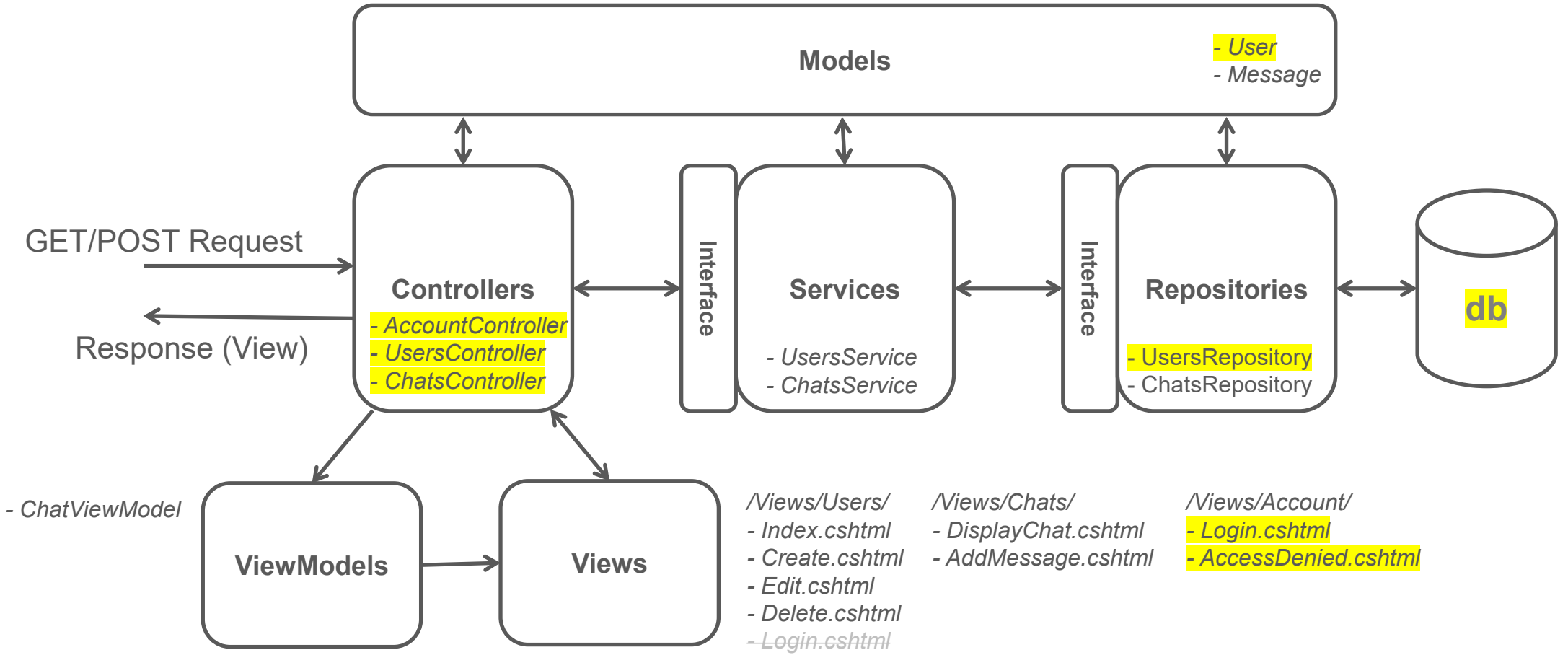
Web Programming 2 program

01 (wk-16)	ViewModels / Passing data to a View
02 (wk-17)	State Management
03 (wk-18)	*break*
04 (wk-19)	Services
05 (wk-20)	**Authentication / Authorization**
06 (wk-21)	Partial views / View Components
07 (wk-22)	…
08 (wk-23)	repetition

---

09 (wk-24)	exams term 4
10 (wk-25)	retake exams term 3
11 (wk-26)	retake exams term 4

Architecture WhatsUp application



Authentication / Authorization

Authentication and authorization

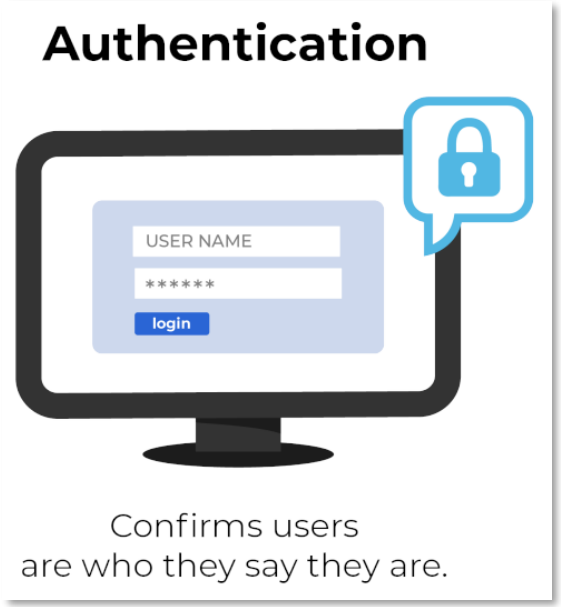
- This presentation is about **authentication** and **authorization**, 2 security concepts that work together to protect the application and its resources (like the database)

- Authentication is the process of **verifying the identity of a user**
- It answers the question: “Is this user really who they claim to be?”

- Authorization is the process of **determining what an authenticated user is allowed to do**
- It answers the question: “Does this user have permission to perform this action?”

Authentication and authorization

- Authentication is the process of **verifying the identity of a user**
- This process is often implemented with a login form (username and password), but might also be done by biometrics (like fingerprint), showing an ID card or passport, …
- When using a login form: if the username and password combination is found in the database, the user is **authenticated** and allowed to access the application

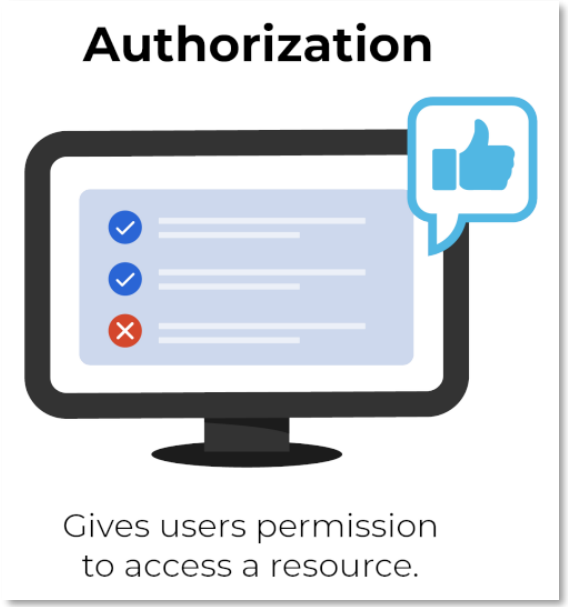


Authentication and authorization

- Authorization is the process of **determining what an authenticated user is allowed to do**

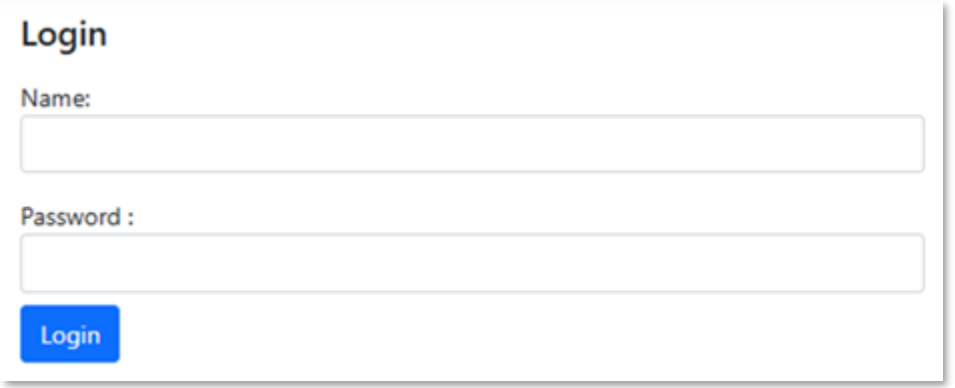
- This means that authentication always comes first
- Granting permission is often implemented through roles
- Role "admin" can do everything, role "guest" can do only basic stuff

- In the WhatsUp application this could be used for managing users, for example: "admin" users can view/create/change/delete users, "guest" users can only view users



Authentication and authorization

- In the WhatsUp application, we already have a login form
- After successful login, the user is authenticated
  (their identity is verified)



- Until now, the application stores the (authenticated) user object in a Session, so the system remembers the user across requests
- During this presentation, we will replace this approach by using a **built-in** authentication and authorization mechanism, which also allows us to manage user permissions more securely and efficiently

Claims-based authentication

Claims, ClaimsIdentity, ClaimsPrincipal

- The WhatsUp application is created with (cross-platform) framework ASP.NET Core
- ASP.NET Core uses **claims-based authentication**
- It involves Claims, ClaimsIdentity and ClaimsPrincipal

- Claims → a piece of information about the user (like name or email address)
- ClaimsIdentity → a collection of Claims
- ClaimsPrincipal → a collection of ClaimsIdentities

Claims, ClaimsIdentity, ClaimsPrincipal

- A **Claim** is a piece of information about the user
- It consists of a Claim type and a value
- It is stored in the form of name-value pair

- A Claim can be anything, for example: Name Claim, Email Claim, Role Claim, PhoneNumber Claim, etcetera

- An application uses the Claims to check whether the user has the rights to access a resource or functionality (e.g., only authenticated users with Role "Admin" are allowed to delete users)

Claims, **ClaimsIdentity**, ClaimsPrincipal

- The **ClaimsIdentity** is a collection of Claims

- A driving license is an example of a ClaimsIdentity
- It contains claims like FirstName, LastName, DateOfBirth, Driving License Number, etcetera

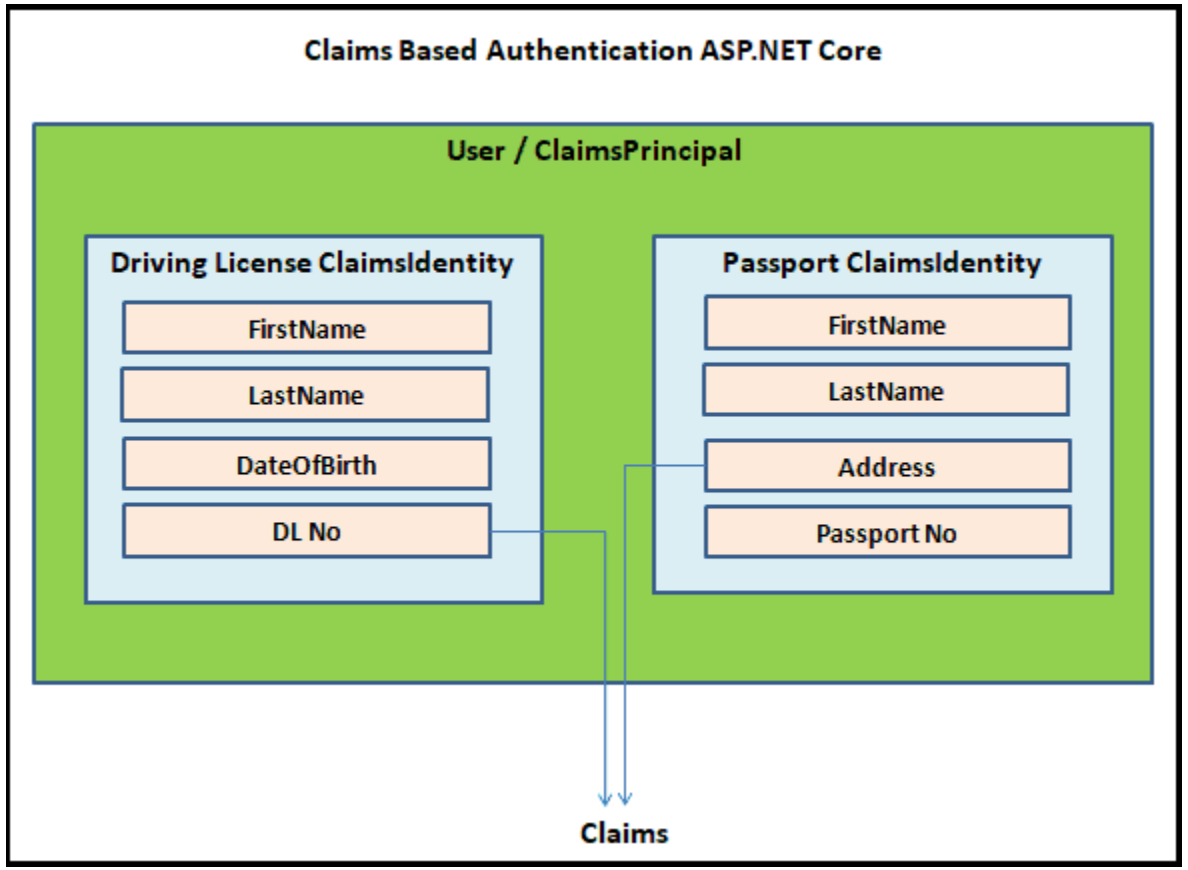
- A passport is another example of a ClaimsIdentity
- It contains claims like FirstName, LastName, Address, Passport Number, etcetera

Claims, ClaimsIdentity, **ClaimsPrincipal**

- **ClaimsPrincipal** contains a collection of ClaimsIdentity
- You can think of the ClaimsPrincipal as the user of your application

- A user can have more than one ClaimsIdentity, for example, both driving license and passport
- Both identifies the same user or ClaimsPrincipal

Claims, ClaimsIdentity, ClaimsPrincipal



Claims, ClaimsIdentity, ClaimsPrincipal

- In our web application, we can access the ClaimsPrincipal (user) through: **HttpContext.User**
- Every request that the web application receives contains the HttpContext object

```
// get the ClaimsPrincipal (user)
ClaimsPrincipal user = HttpContext.User;
```

- By default, this user is not authenticated

- There are several ways for users to authenticate in an application
- Cookie-based authentication and JWT Token-based authentication are the popular ones
- We will use **Cookie-based authentication**

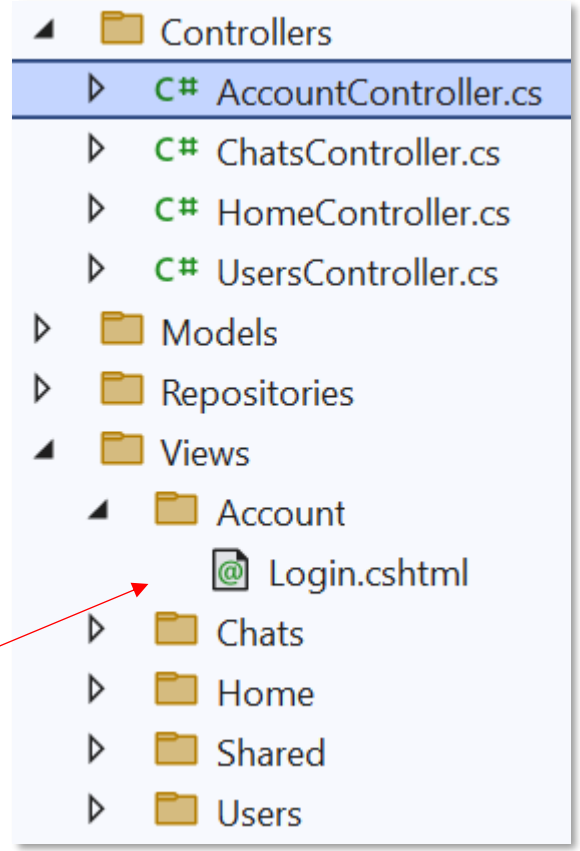
Cookie-based authentication

Adding a separate Account controller

- Before we enable the cookie-based authentication…

- Since we want a better separation of "anything that has to do with Users" (viewing, creating, adding, deleting) and "anything that has to do with authentication" (login and logoff), we will add a new Controller: **AccountController**

- The existing Login and Logoff methods needs to be moved from UsersController → AccountController
- Also move the Login View from /Views/Users → /Views/Account



Enabling cookie-based authentication

- To enable **cookie-based authentication**, we need to add the following code in Program.cs →

```
builder.Services.AddAuthentication(
    CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/AccessDenied";
    });
builder.Services.AddAuthorization();

var app = builder.Build();
```

```
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.UseSession();
```

- This authentication is using a cookie
- The cookie contains the ClaimsPrincipal (the authenticated user) and its claims
- This information is encrypted *(unreadable)* and signed *(changes will be detected)*
- After login, the browser sends the cookie with every request

- The option "LoginPath" set the URL to be used when the user needs to be authenticated
- The option "AccessDeniedPath" set the URL to be used when the user has no access

Restricting access to parts of the application

- After enabling the (cookie-based) authentication, we can specify which parts of the application are only accessible for authorized users
- We do this using the "Authorize" attribute, like in the code below

```
[Authorize]
public class ChatsController : Controller
{
    private readonly IChatsService _chatsService;
    private readonly IUsersService _usersService;

    public ChatsController(IChatsService chatsService, IUsersService usersService)
    {
        _chatsService = chatsService;
        _usersService = usersService;
    }
```

- Putting the "Authorize" attribute above the ChatsController class, makes all methods (AddMessage, DisplayChat, …) unavailable for users that are not authorized
- No specific claims are used here, so it simply means that the user needs to be authenticated

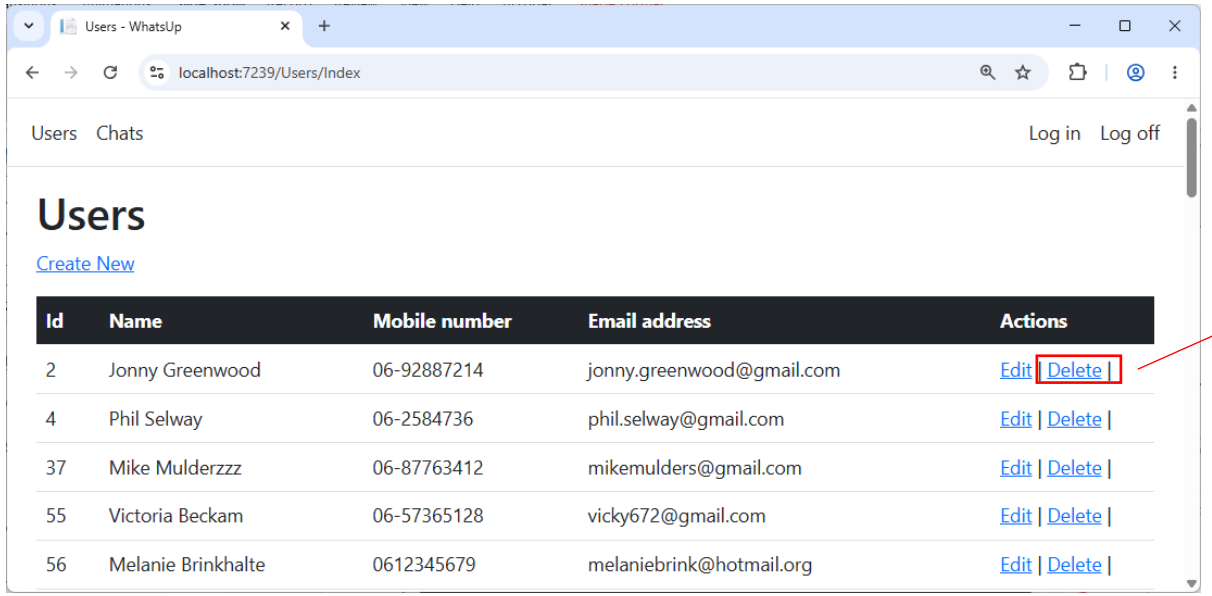
Restricting access to parts of the application

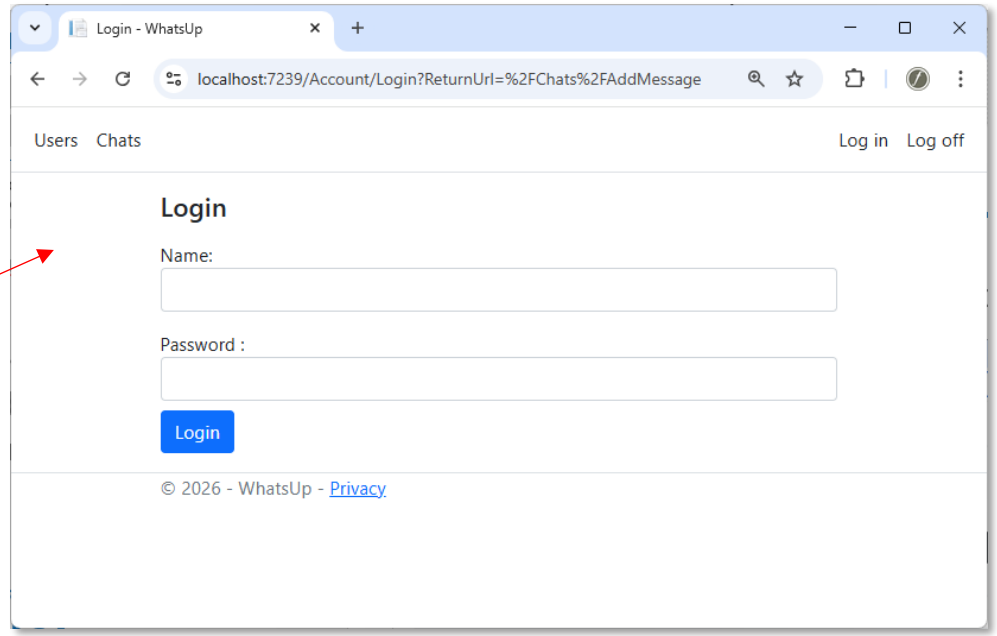
- The "Authorize" attribute can also be used for a specific method, like the Delete method in the UsersController *(see screenshot below)*
- This will keep the other methods (like GetAll and GetById) available for users that are not authenticated

```
// GET: UsersController/Delete/5
[Authorize]
public ActionResult Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    // get user (via service)
    User? user = _usersService.GetById((int)id);
    if (user == null)
    {
        return NotFound();
    }

    return View(user);
}
```

Testing the enabled authentication

– Let's see what happens if we now try to access /Users/Delete





We are automatically redirected to /Account/Login !

Class exercise

- Enable cookie-based authentication in your Whatsup application
- Make the complete ChatsController only available for authenticated users *(using [Authorize)] above the ChatsController class)*
- Make method Delete of the UsersController only available for authenticated users *(using [Authorize)] above the Delete method)*
- Test if you are redirected automatically to the login form when accessing /Users/Delete

Login and Logoff

Login and Logoff methods

- It's time to implement the (new) Login and Logoff methods
- Remember, these 2 methods are now part of the AccountController
  /Account/Login and /Account/Logoff

- Also remember, that ASP.NET Core uses **claims-based authentication**
- So, prepare to start coding with Claims, ClaimsIdentity, and ClaimsPrincipal

Account controller - Login

- To the right, you see the code for the Login method
- The line that is red-marked, needs to be added
- We keep the code to store the logged in user in the Session *(other parts depend on this)*

- So, we have to add a call to a new method **SignInUser** and pass the user (returned by service method GetByLoginCredentials)
- This SignInUser will do the authentication job *(see next slide)*

```
[HttpPost]
public ActionResult Login(LoginModel loginModel)
{
    // get user (via service) matching username and password
    User? user = _usersService.GetByLoginCredentials(
                        loginModel.UserName, loginModel.Password);

    if (user == null)
    {
        // bad login, go back to form
        ViewBag.ErrorMessage = "Bad username/password combination!";
        return View(loginModel);
    }
    else
    {
        SignInUser(user);

        // remember logged in user
        HttpContext.Session.SetObject("LoggedInUser", user);

        // redirect to list of users (via URL /Users/Index)
        return RedirectToAction("Index", "Users");
    }
}
```

Account controller - Login

- First a list of **Claim** items is created
- User values are used to fill this list
- Then a **ClaimsIdentity** (like a passport) is created, using the claims list
- Then the **ClaimsPrincipal** (user) is created, using the ClaimsIdentity
- Finally, the principal (user) is signed in, which creates the authentication cookie

```
private void SignInUser(User user)
{
    List<Claim> claims = new List<Claim>
    {
        // a claim is a piece of information about the user
        new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
        new Claim(ClaimTypes.Name, user.UserName),
        new Claim(ClaimTypes.Email, user.EmailAddress)
    };

    // these claims are placed inside a ClaimsIdentity
    ClaimsIdentity identity = new ClaimsIdentity(
        claims, CookieAuthenticationDefaults.AuthenticationScheme);

    // a ClaimsPrincipal represents the actual logged-in user
    ClaimsPrincipal principal = new ClaimsPrincipal(identity);

    // this will create an encrypted cookie (and add it to the current response)
    HttpContext.SignInAsync(principal);
}
```

Account controller - Logoff

- To the right, you see the code for the Logoff method
- The line that is red-marked, needs to be added
- We keep the code to remove the logged in user from the Session

```
public ActionResult Logoff()
{
    // remove the user's authentication information
    HttpContext.SignOutAsync();

    // remove logged in user from Session
    HttpContext.Session.Remove("LoggedInUser");

    // go back to user list (via Index)
    return RedirectToAction("Index", "Users");
}
```

- `HttpContext.SignOutAsync();` is used to **sign the user out**
- It removes the user's authentication information, so they are no longer recognized as logged in *(the authentication cookie will be invalidated, and the browser will delete the cookie)*
- The user will become **anonymous**

Class exercise

- Create an AccountController and move your Login and Logoff methods (and the Login View)
- Implement the Login and Logoff method so it uses cookie-based authentication

- Make sure that method Delete of the UsersController is only available for authenticated users *(using [Authorize)] above the Delete method)*

- Verify that logged in users (authenticated users) can delete users, and anonymous users can not /Users/Delete

Using roles

Restricting access with Role-Claim

- Now that we can login with claims-based authentication, we can use Claims to restrict access to resources/functionalities

- Suppose we only allow admin-users (Role-claim = "Admin") to delete users (/Users/Delete), we could change the Authorize attribute as follows:

```
[Authorize(Roles = "Admin")]
public ActionResult Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    // get user (via service)
    User? user = _usersService.GetById((int)id);
    if (user == null)
    {
        return NotFound();
    }

    return View(user);
}
```

Restricting access with Role-Claim

- We need to give each user in the WhatsUp application a Role (Admin, Member, ...)
- This means that we have to add a "Role" field in database table Users
- Every existing user in the database should have a role (Admin, Member, …)

Users

	Column Name	Data Type	Allow Nulls
🔑	UserId	int	☐
	UserName	nvarchar(250)	☐
	MobileNumber	nvarchar(50)	☐
	EmailAddress	nvarchar(250)	☐
	Password	nvarchar(50)	☐
	Role	nvarchar(20)	☑
			☐

```
public enum UserRole { None, Member, Admin }

public class User
{
    public int UserId { get; set; }
    public string UserName { get; set; }
    public string MobileNumber { get; set; }
    public string EmailAddress { get; set; }
    public string Password { get; set; }
    public UserRole Role { get; set; }
}
```

- Class User and UsersRepository needs to be changed for this extra field/property (!)

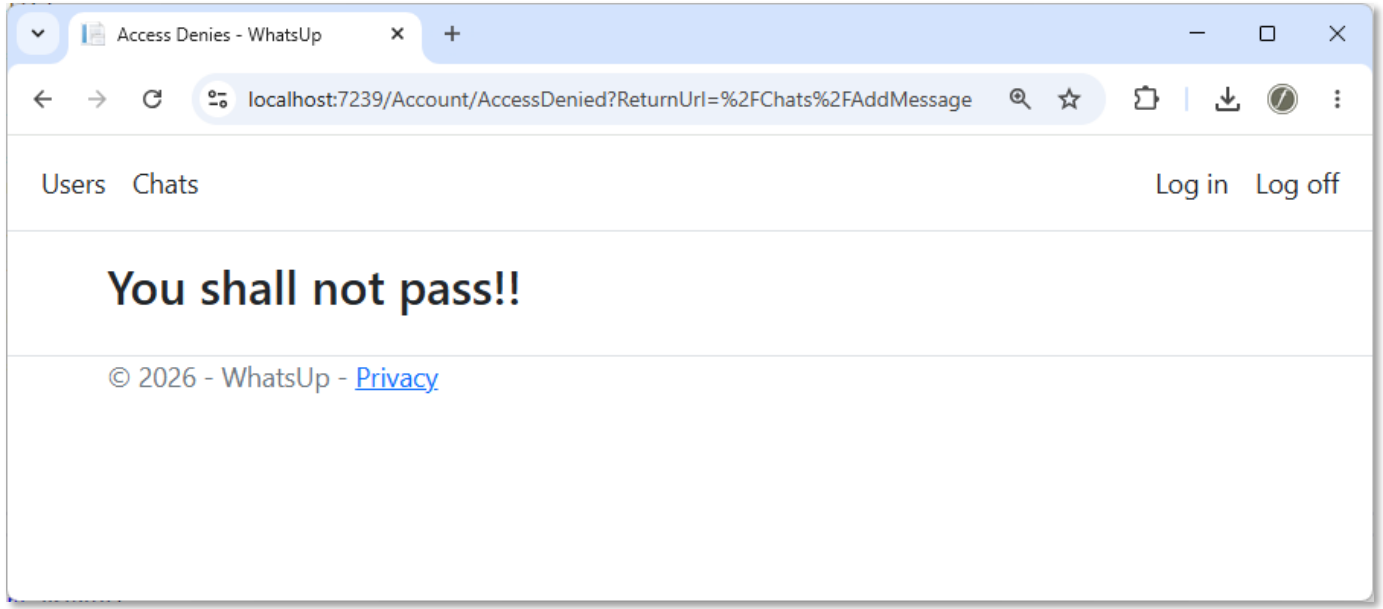
Restricting access with Role-Claim

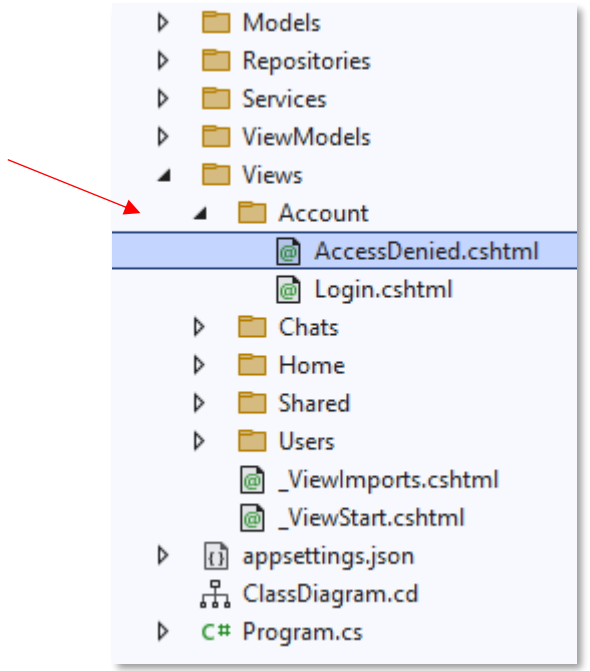
- Now that each user has a role, we can update the **SignInUser** method to make this role a claim
- Be aware, that only claims can be used for restricting access (authorization)

```
private void SignInUser(User user)
{
    List<Claim> claims = new List<Claim>
    {
        // a claim is a piece of information about the user
        new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
        new Claim(ClaimTypes.Name, user.UserName),
        new Claim(ClaimTypes.Email, user.EmailAddress),
        new Claim(ClaimTypes.Role, user.Role.ToString())
    };

    // these claims are placed inside a ClaimsIdentity
    ClaimsIdentity identity = new ClaimsIdentity(
        claims, CookieAuthenticationDefaults.AuthenticationScheme);

    // a ClaimsPrincipal represents the actual logged-in user
    ClaimsPrincipal principal = new ClaimsPrincipal(identity);

    // this will create an encrypted cookie (and add it to the current response)
    HttpContext.SignInAsync(principal);
}
```

Restricting access with Role-Claim

- When authenticated users don't have access, they will be redirected to /Account/AccessDenied
- To make this work, you need a method "AccessDenied" in the AccountController with code "return View();", and of course the View itself → \Views\Account\AccessDenied.cshtml





Class exercise

- Update the UsersController (Delete) to only give access for users with role "Admin“

- Add an extra field Role in database table Users, and give each user a role (Admin, Member, …)
- Add property Role (enumeration UserRole) to class User
- Update class UsersRepository to handle the extra Role field/property

- Update the SignIn-method (in the AccountController) to make this user Role a claim

- Verify that only admin-users have access to the UsersController, and other users will be redirected to the "AccessDenied" View *(make sure this View is available)*

Homework – week 4

- Enable cookie-based authentication in your Whatsup application

- Add an extra field Role in database table Users, and give each user a role (Admin, Guest, …)
- Add property Role (enum UserRole) to class User, and update class UsersRepository to handle this

- Create an AccountController and move your Login and Logoff methods (and the Login View)
- Implement the Login and Logoff method so it uses cookie-based authentication (incl. Role-Claim)

- Update the UsersController so only users with role "Admin“ have access to the Delete method
- Verify that admin-users can access the /Users/Delete, other users are redirected to AccessDenied


holland
hogeschool